Part 2 — Integration Report

StreetRace Manager | Module Logic + Unit Testing + Integration Testing

Generated: 2026-03-22

Command	`/home/rakshit-aggarwal/Desktop/dass_A2/DASS_A2/.venv/bin/python -m pytest -q integration/tests`
Result	**70 passed in 0.08s**
Date	2026-03-22
Test path	`integration/tests/`
Error fixes	3 error-fix commits (see Section 2.4)

2.1 Module Overview

The StreetRace manager system is composed of 10 modules. The table below summarises each one; full logic detail follows.

Module	Functions / Classes	Responsibility
models.py	**Classes:** `StreetRaceError`, `CrewMember`, `Car`… (none)	• Defines the domain model backbone for the full StreetRace system. • Holds canonical in-memory state via StreetRaceState — one shared source of truth across all modules. • Uses dataclasses with slots for clear structure and low-overhead object storage. • Defines StreetRaceError, the unified domain exception for all business-rule validations.

registration.py	`_normalize_text`
	`register_member`
	`require_registered_member`
		• Entry point for onboarding crew members into system state.
		• Normalizes text input, rejects blanks/duplicates, and initializes ranking defaults.
		• Provides require_registered_member so downstream modules enforce preconditions consistently.

crew_management.py	`_normalize_role`
	`members_busy_in_slot`
	`assign_role`
	`set_skill_level`
	+2 more…
		• Adds role and skill management on top of registered members.
		• Validates role normalization and skill ranges (0–10).
		• Implements availability logic by checking scheduling reservations for a slot.
		• Supports role requirement resolution with distinct member assignment across duplicate roles.

inventory.py	`_normalize_text`
	`add_car`
	`require_car`
	`require_available_car`
	+7 more…
		• Manages fleet and resources: cars, spare parts, tools, and cash.
		• Validates all stock/cash operations — no negative quantities, no over-consumption, no negative balance.
		• Tracks car lifecycle transitions (ready ↔ damaged) with severity constraints.
		• Provides strict readiness and existence checks for race/mission/maintenance modules.

scheduling.py	`normalize_slot`
	`_require_event`
	`ensure_no_conflict`
	`schedule_event`
	+2 more…
		• Central reservation engine for race and mission events.
		• Normalizes slot/event data and blocks duplicate schedule IDs.
		• Detects participant and car overlaps in the same slot to prevent impossible plans.
		• Allows incremental assignment of members/cars to existing events with conflict re-validation.

race_management.py	`_normalize_text`
	`_require_race`
	`_require_race_schedule_slot`
	`create_race`
	+2 more…
		• Controls race lifecycle from creation to active start state.
		• Creates races and ensures corresponding schedule entries exist.
		• Validates race entries: registered driver role, available car, uniqueness per driver/car.
		• Re-validates schedule and readiness before transitioning race to active.

mission_planning.py	`_normalize_text`
	`require_mission`
	`plan_mission`
	`start_mission`
	+1 more…
		• Plans and controls non-race mission workflows.
		• Validates mission IDs, types, slots, rewards, and required role lists.
		• Reserves roles and optional cars through crew + scheduling + inventory checks.
		• Supports mission start/complete transitions with strict state and role/car validity checks.

maintenance.py	`_require_repair_mission`
	`_ensure_no_open_repair_for_car`
	`assess_damage`
	`schedule_repair`
	+1 more…
		• Implements repair lifecycle for damaged cars.
		• Converts severity reports into recipe-driven repair requirements (parts/tools/labor).
		• Prevents duplicate open repair missions for the same car.
		• On completion: validates resource sufficiency, consumes parts, charges labor, restores car readiness.

results.py	`_require_race`
	`_slugify`
	`_restore_state`
	`_normalize_damage_reports`
	+3 more…
		• Finalizes races and applies score/economy side effects.
		• Validates placement payload against entered drivers.
		• Updates rankings and prize cash, marks damaged cars, and can auto-schedule repairs.

Module	Functions / Classes	Responsibility
		• Uses snapshot rollback to maintain transactional safety when any step fails.

`cli.py`	`seed_demo_data`
	`_print_summary`
	`run`
		• Provides a demo executable flow that wires all modules together.
		• Seeds representative data and runs race → results → repair → mission chain.
		• Acts as an integration smoke flow validating real-world orchestration behavior.

2.2 Unit-style Module Tests

These test files each target a single module. They verify individual function behaviour, input validation, boundary conditions, and error paths in isolation.

integration/tests/test_registration_module.py — module: registration.py

Test Case	Scenario	Expected	Actual	Result
`test_register_member_stores_name_role_and_ranking`	Register a new member with name, role, and ranking	Member stored with correct fields	Member stored correctly	PASS
`test_register_member_rejects_duplicates`	Attempt to register the same member name twice	StreetRaceError on duplicate	Error raised as expected	PASS
`test_require_registered_member_rejects_unknown_names`	Look up a name that was never registered	StreetRaceError for unknown name	Error raised as expected	PASS

integration/tests/test_crew_management_module.py — module: crew_management.py

Test Case	Scenario	Expected	Actual	Result
`test_assign_role_requires_registration`	Assign role to an unregistered member	StreetRaceError — must be registered first	Error raised as expected	PASS
`test_assign_role_and_skill_update_registered_member`	Assign role and set skill level on registered member	Role and skill updated on member record	Fields updated correctly	PASS
`test_list_available_members_filters_busy_people_for_the_same_slot`	List available members when some are scheduled	Busy members excluded from result	Filtered correctly	PASS
`test_require_role_members_uses_distinct_people_for_duplicate_roles`	Resolve two identical role requirements	Two distinct members assigned (no reuse)	Distinct assignment confirmed	PASS
`test_set_skill_level_validates_level_range`	Set skill level to 0, 10, and out-of-range values	Valid range accepted; out-of-range raises error	Validation works correctly	PASS

integration/tests/test_inventory_module.py — module: inventory.py

Test Case	Scenario	Expected	Actual	Result
`test_add_car_and_require_available_car`	Add a car and retrieve it as available	Car present and available	Car available as expected	PASS
`test_add_car_rejects_duplicates`	Add the same car ID twice	StreetRaceError on second add	Error raised as expected	PASS
`test_cash_balance_cannot_go_negative`	Deduct more cash than current balance	StreetRaceError — insufficient funds	Error raised as expected	PASS
`test_spare_parts_and_tools_are_tracked`	Add parts/tools and verify tracking	Quantities stored and retrievable	Tracked correctly	PASS
`test_damaged_car_is_not_available_until_repaired`	Mark car damaged then	StreetRaceError — car unavailable until repaired	Unavailability enforced correctly	PASS

	require available car			

integration/tests/test_scheduling_module.py — module: scheduling.py

Test Case	Scenario	Expected	Actual	Result
`test_schedule_event_creates_a_reservation`	Create a new schedule event	Event stored in schedule	Event created correctly	PASS
`test_assign_member_to_event_requires_registration`	Assign unregistered member to event	StreetRaceError — not registered	Error raised as expected	PASS
`test_assign_member_to_event_blocks_same_slot_conflicts`	Assign same member to two events in same slot	StreetRaceError — slot conflict	Conflict detected correctly	PASS
`test_assign_car_to_event_blocks_same_slot_conflicts`	Assign same car to two events in same slot	StreetRaceError — car slot conflict	Conflict detected correctly	PASS

integration/tests/test_race_management_module.py — module: race_management.py

Test Case	Scenario	Expected	Actual	Result
`test_create_race_adds_a_race_and_schedule_entry`	Create a new race	Race and schedule entry both created	Both created correctly	PASS
`test_enter_race_allows_only_registered_drivers`	Enter an unregistered driver into a race	StreetRaceError — not a driver	Error raised as expected	PASS
`test_enter_race_reserves_driver_and_car`	Enter valid driver+car into race	Driver and car reserved in schedule	Reservation confirmed	PASS
`test_start_race_requires_at_least_one_entry`	Start a race with no entries	StreetRaceError — no entries	Error raised as expected	PASS
`test_race_actions_require_the_schedule_entry_to_exist`	Operate on a race with missing schedule entry	StreetRaceError — schedule entry missing	Error raised as expected	PASS

integration/tests/test_results_module.py — module: results.py

Test Case	Scenario	Expected	Actual	Result
`test_record_race_result_updates_rankings_and_cash`	Record valid placements for an active race	Rankings and cash updated per placement	Updates applied correctly	PASS
`test_record_race_result_requires_matching_placements`	Submit placements not matching entered drivers	StreetRaceError — placement mismatch	Error raised as expected	PASS
`test_record_race_result_can_plan_a_follow_up_repair`	Record result with damage report and	Repair mission auto-planned after result	Repair planned correctly	PASS

	mechanic available			
`test_record_race_result_rolls_back_if_auto_repair_planning_fails`	Record result but repair planning fails mid-way	Full rollback — state unchanged	Rollback confirmed	**PASS**
`test_record_race_result_can_schedule_repairs_for_multiple_damaged_cars`	Multiple damaged cars in one result	Separate repair missions for each damaged car	Multiple repairs planned	**PASS**
`test_record_race_result_marks_multiple_cars_damaged_without_auto_repairs`	Damage with no mechanic available	Cars marked damaged, no repair missions created	Damage marked without repairs	**PASS**

integration/tests/test_mission_planning_module.py — module: mission_planning.py

Test Case	Scenario	Expected	Actual	Result
`test_plan_mission_assigns_required_roles`	Plan a mission with all required roles available	Mission planned with roles assigned	Roles assigned correctly	**PASS**
`test_plan_mission_rejects_unavailable_roles`	Plan a mission when required role is unavailable	StreetRaceError — role unavailable	Error raised as expected	**PASS**
`test_plan_mission_rejects_damaged_car_for_non_repair_work`	Plan a non-repair mission with a damaged car	StreetRaceError — car not available	Error raised as expected	**PASS**
`test_start_mission_moves_planned_mission_to_active`	Start a previously planned mission	Mission status transitions to active	Status updated correctly	**PASS**

integration/tests/test_maintenance_module.py — module: maintenance.py

Test Case	Scenario	Expected	Actual	Result
`test_assess_damage_creates_repair_mission_from_recipe`	Assess a damaged car with known severity level	Repair mission created with correct requirements	Mission created correctly	**PASS**
`test_schedule_repair_rejects_missing_parts`	Schedule repair but spare parts are missing	StreetRaceError — parts unavailable	Error raised as expected	**PASS**
`test_complete_repair_consumes_resources_and_restores_car`	Complete a fully scheduled repair	Parts consumed, labor charged, car marked ready	All side effects applied correctly	**PASS**

integration/tests/test_module_completeness.py — module: All modules (completeness sweep)

Test Case	Scenario	Expected	Actual	Result
`test_registration_module_validates_required_text`	Blank/whitespace inputs to registration	StreetRaceError on blank/invalid text	Error raised	**PASS**

`test_crew_management_validates_blank_inputs_and_empty_role_requests`	Blank role names and empty role request lists	Errors raised for blank/empty inputs	Errors raised	**PASS**
`test_crew_management_handles_optional_and_invalid_schedule_slots`	None slot and invalid slot formats	Handled without crash / errors raised	Handled correctly	**PASS**
`test_inventory_module_covers_validation_branches`	Edge cases for all inventory validations	All boundary conditions raise correct errors	All pass	**PASS**
`test_scheduling_module_validates_inputs_and_missing_events`	Invalid slot data and missing event references	Errors raised appropriately	Errors raised	**PASS**
`test_race_management_rejects_invalid_setup_and_duplicate_entries`	Duplicate race IDs and invalid setup	StreetRaceError on duplicates/invalid setup	Errors raised	**PASS**
`test_race_management_validates_race_state_before_start_and_entry`	Enter/start race in wrong state	StreetRaceError on wrong state	Errors raised	**PASS**
`test_race_management_rechecks_driver_roles_before_start`	Role removed after entry, then start attempted	StreetRaceError — role check fails at start	Error raised	**PASS**
`test_results_module_validates_missing_and_inactive_races`	Record result for nonexistent/inactive race	StreetRaceError	Error raised	**PASS**
`test_results_module_preserves_state_when_result_input_is_invalid`	Invalid placement data	State unchanged after bad input	Rollback confirmed	**PASS**
`test_results_module_covers_zero_prize_branch_and_sorted_rankings`	Zero-prize race and ranking sort	Zero prize handled; rankings sorted correctly	Correct	**PASS**
`test_mission_planning_validates_setup_and_start_requirements`	Missing/invalid fields at plan and start	Errors raised for invalid setups	Errors raised	**PASS**
`test_mission_planning_start_and_completion_cover_remaining_branches`	Start non-planned / complete non-active mission	Errors raised on wrong-state transitions	Errors raised	**PASS**
`test_mission_planning_covers_non_planned_and_zero_reward_completion`	Complete with zero reward	Zero reward handled gracefully	Handled correctly	**PASS**
`test_maintenance_module_validates_assessment_and_repair_planning`	Assess unknown/already-damaged car	Errors raised for invalid states	Errors raised	**PASS**
`test_maintenance_module_validates_custom_repair_requirements`	Custom repair parts/tools requirements	Custom requirements stored and checked	Correct	**PASS**
`test_maintenance_completion_validates_mission_shape_and_resources`	Complete repair with wrong mission type	StreetRaceError — mission shape invalid	Error raised	**PASS**
`test_maintenance_completion_covers_invalid_notes_and_cash_rules`	Notes validation and cash deduction rules	Invalid notes and negative cash raise errors	Errors raised	**PASS**
`test_maintenance_completion_supports_zero_labor_repairs`	Repair with zero labor cost	Completes without charging labor	Handled correctly	**PASS**

2.3 Cross-Module Integration Flows

These 16 tests in test_integration_flows.py exercise multiple modules collaborating in realistic end-to-end scenarios. Each row shows the module chain, scenario, and outcome.

Modules covered: registration.py • crew_management.py • inventory.py • scheduling.py • race_management.py • results.py • mission_planning.py • maintenance.py • cli.py

integration/tests/test_integration_flows.py — 16 cross-module flow tests

Test Case	Modules Exercised	Scenario	Expected	Result
`test_register_driver_then_enter_the_driver_into_a_race`	registration.py → race_management.py	Register member, assign driver role, enter race	Race entry created	PASS
`test_assigning_driver_role_after_registration_unlocks_race_entry`	registration.py → crew_management.py → race_management.py	Register, then assign role, then enter race	Race entry succeeds after role assigned	PASS
`test_attempting_to_enter_a_race_without_a_registered_driver_fails`	registration.py → race_management.py	Enter race with unregistered name	StreetRaceError	PASS
`test_completing_a_race_updates_results_and_inventory_cash`	race_management.py → results.py → inventory.py	Full race flow: create, enter, start, record result	Rankings and cash updated	PASS
`test_race_reservation_blocks_same_slot_delivery_with_the_same_car`	scheduling.py → race_management.py → mission_planning.py	Car reserved in race slot, then delivery for same slot	StreetRaceError — car conflict	PASS
`test_active_mission_blocks_same_slot_race_entry_for_the_busy_driver`	mission_planning.py → scheduling.py → race_management.py	Driver in active mission, then enters race in same slot	StreetRaceError — driver conflict	PASS
`test_race_workflows_fail_cleanly_if_the_schedule_entry_is_missing`	race_management.py → scheduling.py	Race operations without prior schedule entry	StreetRaceError — schedule missing	PASS
`test_damaged_car_flow_requires_a_mechanic_before_repair_mission_proceeds`	inventory.py → maintenance.py → crew_management.py	Damage car, attempt repair without mechanic	StreetRaceError — mechanic unavailable	PASS
`test_completed_race_and_repair_flow_updates_inventory_resources`	race_management.py → results.py → maintenance.py → inventory.py	Race result triggers repair, repair consumes parts	Parts consumed, car restored	PASS
`test_damaged_car_cannot_receive_a_second_unfinished_repair_mission`	inventory.py → maintenance.py	Open repair exists, attempt second repair on same car	StreetRaceError — duplicate repair	PASS
`test_repair_mission_cannot_start_after_the_car_was_already_fixed`	maintenance.py → inventory.py	Car repaired, attempt to start already-closed repair mission	StreetRaceError — car already fixed	PASS
`test_failed_auto_repair_planning_rolls_back_the_race_result`	results.py → maintenance.py	Race result triggers repair planning	Full rollback of race result	PASS

		which fails mid-way		
`test repaired car can complete a later delivery mission for extra reward`	maintenance.py → mission_planning.py → inventory.py	Repair car, then use it for a delivery mission	Mission completes, reward applied	**PASS**
`test_repaired_car_and_driver_can_reenter_a_later_race_and_stack_rewards`	maintenance.py → race_management.py → results.py	Repair car, re-enter race, record result	Stacked rewards applied correctly	**PASS**
`test_missions_cannot_start_if_required_roles_are_unavailable`	crew_management.py → mission_planning.py	All members with required role are busy	StreetRaceError — roles unavailable	**PASS**
`test_cli_run_executes_the_demo_flow_end_to_end`	cli.py (all modules)	cli.run() seeds data and executes full race+mission chain	No errors, summary printed	**PASS**

2.4 Part 2 Error-Fix Evidence

Three error-fix commits addressed logical defects discovered through integration testing. Each entry documents the defect, affected functions, fix made, and the tests that prove correctness.

Error 1 — Harden the StreetRace module rules

Commit Hash	`95e6166c5ed8dd2deeba9cd1abe8d45dea0b4ab4`
Defect Found	Multiple modules lacked strict validation guards: missing race-state checks, missing slot validation in scheduling, missing type guards in mission planning.

Affected Functions	`schedule_event (scheduling.py)`
	`create_race, enter_race (race_management.py)`
	`plan_mission (mission_planning.py)`
	`record_race_result (results.py)`

Fix Proven By	test_scheduling_module_validates_inputs_and_missing_events, test_race_management_validates_race_state_before_start_and_entry, test_mission_planning_validates_setup_and_start_requirements all pass after fix.

Files Changed	`integration/code/streetrace_manager/mission_planning.py`
	`integration/code/streetrace_manager/race_management.py`
	`integration/code/streetrace_manager/results.py`
	`integration/code/streetrace_manager/scheduling.py`
	`integration/diagrams/streetrace_call_graph.mmd`

Error 2 — Guard race operations with schedule checks

Commit Hash	`5bd19744217a7f3d4d97249cf31f692041d2d022`
Defect Found	Race operations (enter_race, start_race) did not verify the schedule entry existed before proceeding, allowing races to enter/start in an inconsistent state.

Affected Functions	`enter_race (race_management.py)`
	`start_race (race_management.py)`
	`_require_race_schedule_slot (race_management.py)`

Fix Proven By	test_race_actions_require_the_schedule_entry_to_exist and test_race_workflows_fail_cleanly_if_the_schedule_entry_is_missing both pass after fix.
Files Changed	`integration/code/streetrace_manager/race_management.py` `integration/tests/test_race_management_module.py`

Error 3 — Roll back partial race result updates

Commit Hash	`2d8c904db9e1d8a4df4818d4c2c287905c4fb602`
Defect Found	record_race_result applied ranking/cash updates before validating all steps; if auto-repair planning failed mid-way, partial state mutations were left in place.

Affected Functions	`record_race_result (results.py)`
	`_restore_state (results.py)`
	`_apply_result_updates (results.py)`

Fix Proven By	test_record_race_result_rolls_back_if_auto_repair_planning_fails confirms that state is fully restored after a partial failure.
Files Changed	`integration/code/streetrace_manager/results.py` `integration/tests/test_results_module.py`

Conclusion: Part 2 is module-complete with detailed unit-level validation, full cross-module integration test coverage, and all 3 logical defects resolved. Suite: 70 passed in 0.08s.